MLP Coursework 1

S2890639

Abstract

In this report we study the problem of overfitting, which is the training regime where performance increases on the training set but decrease on validation data. Overfitting prevents our trained model from generalizing well to unseen data. We first analyse the given example and discuss the probable causes of the underlying problem. Then we investigate how the depth and width of a neural network can affect overfitting in a feedforward architecture and observe that increasing width and depth tend to enable further overfitting. Next we discuss how two standard methods, Dropout and Weight Penalty, can mitigate overfitting, then describe their implementation and use them in our experiments to reduce the overfitting on the EMNIST dataset. Based on our results, we ultimately find that both dropout and weight penalty are able to mitigate overfitting.

We further explore alternative loss and activation functions, and evaluate whether their performance relates to problems of overfitting.

Finally, we conclude the report with our observations and related work. Our main findings indicate that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

1. Introduction

In this report we focus on a common and important problem while training machine learning models known as overfitting, or overtraining [1], which is the training regime where performances increase on the training set but decrease on unseen data. We first start with analysing the given problem in Figure 1, study it in different architectures and then investigate different strategies to mitigate the problem. In particular, Section 2 identifies and discusses the given problem, and investigates the effect of network width and depth in terms of generalization gap (see Ch. 5 in Goodfellow et al. 2016) and generalization performance. Section 3 introduces two regularization techniques to alleviate overfitting: Dropout (Srivastava et al., 2014) and L1/L2 Weight Penalties (see Section 7.1 in Goodfellow et al. 2016). We first explain them in detail and discuss why they are used for alleviating overfitting. We extend our study to implement a combined L1 and L2 penalty.

In Section 4, we incorporate each of them and their various combinations to a three hidden layer[2] neural network, train it on the EMNIST dataset, which contains 131,600 images of characters and digits, each of size 28x28, from 47 classes. We evaluate them in terms of generalization gap and performance, and discuss the results and effectiveness of the tested regularization strategies. Our results show that both dropout and weight penalty are able to mitigate overfitting. We end Section 4, by testing an alternative activation function, and explore whether the observed performance degradation relates to overfitting or some other problem.

Finally, we conclude our study in Section 5, noting that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

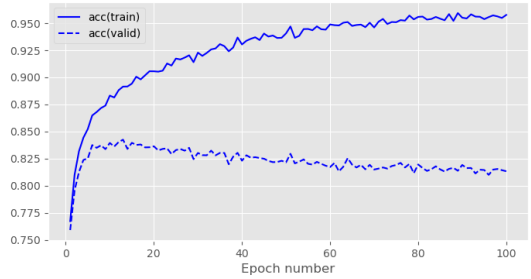
2. Problem identification

Overfitting to training data is a very common and important issue that needs to be dealt with when training neural networks or other machine learning models in general (see Ch. 5 in Goodfellow et al. 2016). A model is said to be overfitting when as the training progresses, its performance on the training data keeps improving, while its is degrading on validation data. Effectively, the model stops learning related patterns for the task and instead starts to memorize specificities of each training sample that are irrelevant to new samples. Overfitting leads to bad generalization performance in unseen data, as performance on validation data is indicative of performance on test data and (to an extent) during deployment.

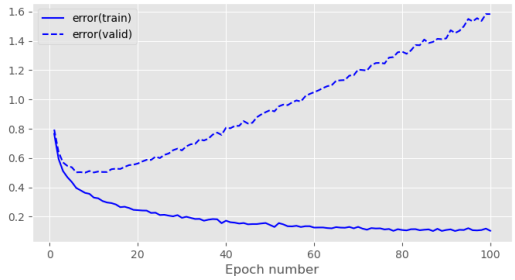
Although it eventually happens to all gradient-based training, it is most often caused by models that are too large with respect to the amount and diversity of training data. The more free parameters the model has, the easier it will be to memorize complex data patterns that only apply to a restricted amount of samples. A prominent symptom of overfitting is the generalization gap, defined as the difference between the validation and training error. A steady increase in this quantity is usually interpreted as the model

[1] Technically, overtraining is the act of training beyond a point at which performance degrades; while overfitting is training to the extent that the learnt function is more complex than required to capture the training data. They correlate, but they are not, strictly speaking, the same thing. To keep things simple, and according to common usage of the terms, we are using them interchangeably in this document, and referring to their joint effect.

[2] We denote all layers as hidden except the final (output) one. This means that depth of a network is equal to the number of its hidden layers + 1.



(a) accuracy by epoch



(b) error by epoch

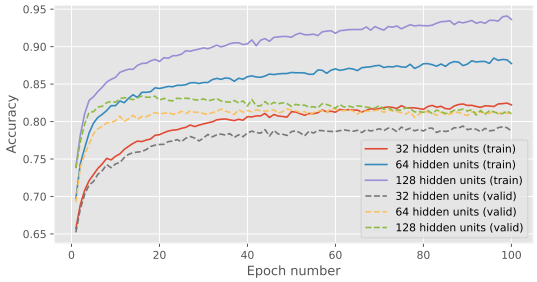
Figure 1. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for the baseline model.

entering the overfitting regime.

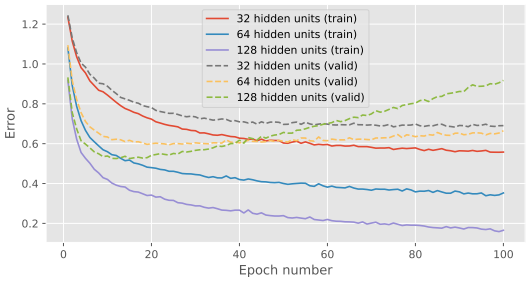
Figure 1a and 1b show a prototypical example of overfitting. We see in Figure 1a that **[Question 1 - Explain what these figures contain and how the curves evolve, and spot where overfitting occurs. Reason based on the min/max points and velocities (direction and magnitude of change) of the accuracy and error curves]** early in training (epochs 0-5), both training and validation accuracies in Figure 1a have high positive velocities, rising steeply. This behavior is mirrored inversely in Figure 1b by training and validation errors, which both have large negative velocities at this stage, indicating rapid fit. From around epoch 10, the validation accuracy curves show smaller magnitudes and their velocities decay toward zero as learning saturates. This flattening is visible in Figure 1a, where the validation curve's slope decreases steadily between epochs 10-18 before reaching its peak. In contrast, the training accuracy slope decreases but remains consistently positive throughout this period, indicating continued improvement. Around epoch 20, the velocity of validation accuracy crosses from positive to negative (the curve peaks), while the validation error velocity crosses from negative to positive (the curve bottoms out). This mirrors the accuracy behaviour exactly: in Figure 1b, validation error reaches its minimum at the same point the validation accuracy reaches its maximum. Meanwhile, training velocities in both Figures remain favorable. This divergence of training and validation performance, when the validation slopes change in sign and the training slopes remains favourable, is what indicates overfitting..

# Hidden Units	Val. Acc.	Train Error	Val. Error
32	70.5	0.889	1.030
64	74.90	0.592	1.070
128	75.7	0.488	1.480

Table 1. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network widths on the EMNIST dataset.



(a) accuracy by epoch



(b) error by epoch

Figure 2. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network widths.

The extent to which our model overfits depends on many factors. For example, the quality and quantity of the training set and the complexity of the model. If we have sufficiently many diverse training samples, or if our model contains few hidden units, it will in general be less prone to overfitting. Any form of regularization will also limit the extent to which the model overfits.

2.1. Network width

[Question Table 1 - Fill in Table 1 with the results from your experiments varying the number of hidden units.] [Question Figure 2 - Replace the images in Figure 2 with figures depicting the accuracy and error, training and validation curves for your experiments varying the number of hidden units.]

First we investigate the effect of increasing the number of hidden units in a single hidden layer network when training on the EMNIST dataset. The network is trained using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 100, for a total of 100 epochs.

The input layer is of size 784, and output layer consists of 47 units. Three different models were trained, with a single hidden layer of 32, 64 and 128 ReLU hidden units respectively. Figure 2 depicts the error and accuracy curves over 100 epochs for the model with varying number of hidden units. Table 1 reports the final accuracy, training error, and validation error. We observe that **[Question 2 - Present your network width experiment results by using the relevant figure and table]** it shows that the validation accuracy improves only marginally with increased network width—rising by 6.2% from 32 to 64 units and by 1.13% from 64 to 128 units. As seen in Figure 2a, the accuracy curves for wider networks exhibit a *higher positive velocity* during the first 10–15 epochs, rising more steeply than the 32-unit model and indicating faster initial learning. After this early phase, the *validation accuracy velocities* for the 64- and 128-unit models gradually decline towards zero as the curves flatten, and eventually *change sign* around epoch 18 when the validation curves peak. In contrast, the 32-unit model's validation curve maintains a small positive velocity for longer and peaks later, around epoch 25, before plateauing.

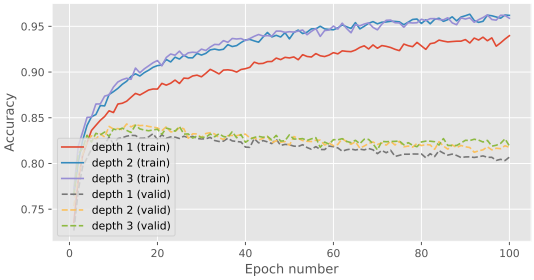
A similar velocity pattern is seen in Figure 2b. The training error curves for the wider models display a *large negative velocity* through epochs 10–15, reflecting rapid error reduction. Their validation error curves also initially have negative velocity but reach their minimum earlier, after which their velocities *change from negative to positive*, indicating that the model begins to overfit. The 32-unit model shows a more gradual decay in validation error velocity that approaches zero without sharply reversing.

Numerically, training error decreases by 33.4% and 17.6% when increasing the width from 32→64 and 64→128 units, respectively, but validation error increases by 38.8% when going from 32→64 units, and 38.3% when going from 64→128 units. This widening generalisation gap, together with the opposing validation and training velocities, demonstrates diminishing returns at higher widths: the additional capacity drives continued improvement on the training set while harming validation performance.

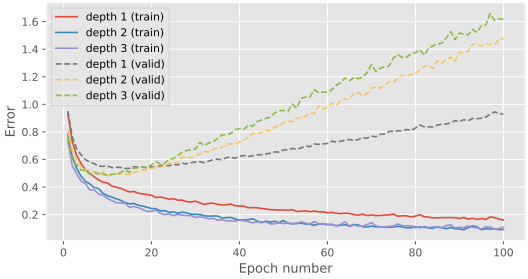
[Question 3 - Discuss whether varying width affects the results in a consistent way, and whether the results are expected and match well with the prior knowledge (by which we mean your expectations as are formed from the relevant Theory and literature)] Increasing network width consistently improves training performance, as reflected by lower error and faster convergence. This aligns with theoretical expectations that wider networks have greater capacity to model complex functions. However, the minimal gains in validation accuracy and the rise in validation error at higher widths indicate overfitting, consistent with prior findings that overly large models can memorize training data without improving generalization. Overall, the results align with theoretical predictions while illustrating the trade-off between model capacity and generalization. .

# Hidden Layers	Val. Acc.	Train Error	Val. Error
1	81.1	0.165	0.918
2	81.8	0.091	1.480
3	81.9	0.096	1.620

Table 2. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network depths on the EMNIST dataset.



(a) accuracy by epoch



(b) error by epoch

Figure 3. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network depths.

2.2. Network depth

[Question Table 2 - Fill in Table 2 with the results from your experiments varying the number of hidden layers.] [Question Figure 3 - Replace these images with figures depicting the accuracy and error, training and validation curves for your experiments varying the number of hidden layers.]

Next we investigate the effect of varying the number of hidden layers in the network. Table 2 and Figure 3 depict results from training three models with one, two and three hidden layers respectively, each with 128 ReLU hidden units. As with previous experiments, they are trained with the Adam optimizer with a learning rate of 9×10^{-4} and a batch size of 100.

We observe that **[Question 4 - Present your network depth experiment results by using the relevant figure and table]** validation accuracy improves modestly with increased network depth, rising by 0.8% from 1 to 2 layers and by 0.1% from 2 to 3 layers. Training curves show slightly faster convergence and lower training error, decreas-

ing by 45% and 5% across these increments, respectively. However, validation error rises by 61% between 1 and 2 layers and by 9% between 2 and 3 layers, suggesting diminishing returns and overfitting at higher depths. This trend is reflected in Figure 3a and Figure 3b, where training performance continues to improve while validation performance plateaus and diverges after approximately 20 epochs.

.

[Question 5 - Discuss whether varying depth affects the results in a consistent way, and whether the results are expected and match well with the prior knowledge (by which we mean your expectations as are formed from the relevant Theory and literature)] Increasing network depth yields consistent improvements in training performance, which aligns with theoretical expectations that deeper architectures can represent more complex mappings. However, the limited gains in validation accuracy and the rising validation error indicate overfitting, consistent with prior literature suggesting that deeper networks require additional regularization or data to generalize effectively. Overall, the observed results match expected theoretical trends, showing the balance between representational capacity and generalization.

.

3. Regularization

In this section, we investigate three regularization methods to alleviate the overfitting problem, specifically dropout layers, the L1 and L2 weight penalties and label smoothing.

3.1. Dropout

Dropout (Srivastava et al., 2014) is a stochastic method that randomly inactivates neurons in a neural network according to an hyperparameter, the inclusion rate (*i.e.* the rate that an unit is included). Dropout is commonly represented by an additional layer inserted between the linear layer and activation function. Its forward pass during training is defined as follows:

$$\text{mask} \sim \text{bernoulli}(p) \tag{1}$$

$$\boldsymbol{y}' = \text{mask} \odot \boldsymbol{y} \tag{2}$$

where $\boldsymbol{y}, \boldsymbol{y}' \in \mathbb{R}^d$ are the output of the linear layer before and after applying dropout, respectively. mask $\in \mathbb{R}^d$ is a mask vector randomly sampled from the Bernoulli distribution with inclusion probability p, and $\odot$ denotes the element-wise multiplication.

At inference time, stochasticity is not desired, so no neurons are dropped. To account for the change in expectations of the output values, we scale them down by the inclusion probability p:

$$\boldsymbol{y}' = \boldsymbol{y} * p \tag{3}$$

As there is no nonlinear calculation involved, the backward propagation is just the element-wise product of the gradients with respect to the layer outputs and mask created in the forward calculation. The backward propagation for dropout is therefore formulated as follows:

$$\frac{\partial \boldsymbol{y}'}{\partial \boldsymbol{y}} = mask \tag{4}$$

Dropout is an easy to implement and highly scalable method. It can be implemented as a layer-based calculation unit, and be placed on any layer of the neural network at will. Dropout can reduce the dependence of hidden units between layers so that the neurons of the next layer will not rely on only few features from of the previous layer. Instead, it forces the network to extract diverse features and evenly distribute information among all features. By randomly dropping some neurons in training, dropout makes use of a subset of the whole architecture, so it can also be viewed as bagging different sub networks and averaging their outputs.

3.2. Weight penalty

L1 and L2 regularization (Ng, 2004) are simple but effective methods to mitigate overfitting to training data. The application of L1 and L2 regularization strategies could be formulated as adding penalty terms with L1 and L2 norm square of weights in the cost function without changing other formulations. The idea behind this regularization method is to penalize the weights by adding a term to the cost function, and explicitly constrain the magnitude of the weights with either the L1 and L2 norms. The optimization problem takes a different form:

$$\text{L1: } \min_{\boldsymbol{w}} E_{\text{data}}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w}) + \lambda \|w\|_1 \tag{5}$$

$$\text{L2: } \min_{\boldsymbol{w}} E_{\text{data}}(\boldsymbol{X}, \boldsymbol{y}, \boldsymbol{w}) + \lambda \|w\|_2^2 \tag{6}$$

where E_{data} denotes the cross entropy error function, and $\{\boldsymbol{X}, \boldsymbol{y}\}$ denotes the input and target training pairs. λ controls the strength of regularization.

Weight penalty works by constraining the scale of parameters and preventing them to grow too much, avoiding overly sensitive behaviour on unseen data. While L1 and L2 regularization are similar to each other in calculation, they have different effects. Gradient magnitude in L1 regularization does not depend on the weight value and tends to bring small weights to 0, which can be used as a form of feature selection, whereas L2 regularization tends to shrink the weights to a smaller scale uniformly.

3.2.1. Combination of L1 and L2 Regularisation

[Question 6 - Explain how one could use a combination of L1 and L2 regularisation. Discuss any potential benefits of this approach. Present an equation for this combined loss function and explain all relevant variables and hyperparameters.] According to Zou & Hastie 2005, combining L1 and L2 regularisation can leverage the strengths of both methods to improve model performance. This technique is often referred to as Elastic Net regularisa-

tion. The combined loss function can be expressed as:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2 \tag{7}$$

where:

- $\mathcal{L}$ is the total loss function.
- $\mathcal{L}_{\text{data}}$ is the original loss function (e.g., cross-entropy loss).
- w_i represents the weights of the model.
- λ_1 is the hyperparameter controlling the strength of L1 regularisation, promoting sparsity in the model weights.
- λ_2 is the hyperparameter controlling the strength of L2 regularisation, encouraging smaller weight values and reducing overfitting.

The potential benefits of this approach include:

- Sparsity: L1 regularisation can lead to sparse models by driving some weights to zero, effectively performing feature selection.
- Stability: L2 regularisation helps to stabilize the model by penalizing large weights, which can improve generalization.
- Flexibility: The combination allows for a balance between sparsity and stability, which can be tuned based on the specific dataset and task.

In our implementation, we can use a mixing parameter α to control the relative contributions of L1 and L2 regularisation:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \left(\alpha \sum_i |w_i| + (1 - \alpha) \sum_i w_i^2 \right) \tag{8}$$

where λ is the overall regularisation strength, and α (ranging from 0 to 1) determines the balance between L1 and L2 penalties. By controlling α, we can adjust the model to prioritize either sparsity or stability, depending on the requirements of the task at hand.

.

3.3. Label smoothing

Label smoothing regularizes a model based on a softmax with K output values by replacing the hard target 0 labels and 1 labels with $\frac{\alpha}{K-1}$ and $1 - \alpha$ respectively. α is typically set to a small number such as 0.1.

$$\begin{cases} \frac{\alpha}{K-1}, & \text{if} \quad t_k = 0 \\ 1 - \alpha, & \text{if} \quad t_k = 1 \end{cases} \tag{9}$$

The standard cross-entropy error is typically used with these *soft* targets to train the neural network. Hence, implementing label smoothing requires only modifying the targets of training set. This strategy may prevent a neural network to obtain very large weights by discouraging very high output values.

4. Balanced EMNIST Experiments

[Question Table 3 - Fill in Table 3 with the results from your experiments for the missing hyperparameter values for each of L1 regularisation, L2 regularisation, Dropout and label smoothing (use the values shown on the table).]

[Question Figure 4 - Replace these images with figures depicting the Validation Accuracy and Generalisation Gap (difference between validation and training error) for each of the experiment results varying the Dropout inclusion rate, and L1/L2 weight penalty depicted in Table 3 (including any results you have filled in).]

[Question Table 4 - Results from 4 training runs varying L1+L2 regularisation.]

[Question Figure 5 - Replace these images with figures depicting the Validation Accuracy, Training Accuracy, and Generalisation Gap (difference between validation and training performance) for each of the experiment results varying the Dropout inclusion rate and L1/L2 weight penalty depicted in Table 3 (including any results you have filled in).]

[Question Figure 6 - Add figures depicting the Training Accuracy, Error, and Gradient Flow Across Layers for the 1 experiment run with the Custom Activation function (produce these by running the "Problematic Training" cell in the Coursework_1.ipynb notebook).]

Here we evaluate the effectiveness of the given regularization methods for reducing the overfitting on the EMNIST dataset. We build a baseline architecture with three hidden layers, each with 128 neurons, which suffers from overfitting as shown in section 2.

Here we train the network with a lower learning rate of 10^{-4}, as the previous runs were overfitting after only a handful of epochs. Results for the new baseline (c.f. Table 3) confirm that lower learning rate helps, so all further experiments are run using it.

Here, we apply the L1 or L2 regularization with dropout to our baseline and search for good hyperparameters on the validation set. Then, we apply the label smoothing with $\alpha = 0.1$ to our baseline. We summarize all the experimental results in Table 3. For each method except the label smoothing, we plot the relationship between generalization gap and validation accuracy in Figure 5.

First we analyze three methods separately, train each over a set of hyperparameters and compare their best performing results.

[Question 7 - Assume you had a limited experiment budget for exactly 4 experiment runs dedicated to testing the effectiveness of your proposed combination of L1

Model	Hyperparameter value(s)	Validation accuracy	Train Error	Validation Error
Baseline	-	83.7	0.241	0.533
Dropout	0.6	80.7	0.549	0.593
	0.7	83.1	0.448	0.509
	0.85	85.1	0.329	0.434
	0.97	***85.4***	***0.244***	***0.457***
L1 penalty	5e-4	*79.5*	*0.642*	*0.658*
	1e-3	74.3	0.879	0.880
	5e-3	2.41	3.850	3.850
	5e-2	2.20	3.850	3.850
L2 penalty	5e-4	*85.1*	*0.306*	*0.460*
	1e-3	84.8	0.358	0.453
	5e-3	81.3	0.586	0.607
	5e-2	39.2	2.258	2.256
Label smoothing	0.1	84.9	1.02	1.16

Table 3. Results of all hyperparameter search experiments. *italics* indicate the best results per series (Dropout, L1 Regularisation, L2 Regularisation, Label smoothing) and **bold** indicates the best overall.

Run	λ / α	Val. Acc. (%)	Train Error	Val. Error
1	1×10^{-5} / 0.900	84.3	0.263	0.503
2	1×10^{-5} / 0.750	84.2	0.251	0.495
3	1×10^{-5} / 0.500	83.8	0.248	0.517
4	1×10^{-5} / 0.250	83.9	0.258	0.517

Table 4. Validation accuracy (%) and training/validation cross-entropy error for four runs with a fixed regularisation strength $\lambda = 1 \times 10^{-5}$ and varying Elastic Net mixing parameter α.

and L2 Regularization in Section 3.2.1. Argue for which 4 configurations to run based on any previous results or other information you consider relevant. Run those experiments and add the results in Table 4 and Figure 5 (you will then discuss those results alongside all others in the next question below).

(Even if you have not managed to implement your L1 and L2 combination; still argue based on your proposed approach).] Analysis on prior L1 and L2 results from 3 suggests optimal hyperparameters around 5×10^{-4} for L1 and 5×10^{-4} for L2 by suggesting that lower regularisation strengths yield better validation accuracy. To explore this further within our limited budget of 4 experiment runs, we select a fixed regularization strength of $\lambda = 1 \times 10^{-5}$, slightly lower than our best individual penalties to follow the trend of small strengths. We then vary the mixing parameter α across a range from L1-dominant to L2-dominant configurations to assess the impact of the balance between the two penalties. Our selected configurations are as follows:

- Run 1: $\lambda = 1 \times 10^{-5}$, $\alpha = 0.25$ (L2-dominant)
- Run 2: $\lambda = 1 \times 10^{-5}$, $\alpha = 0.5$ (Balanced)
- Run 3: $\lambda = 1 \times 10^{-5}$, $\alpha = 0.75$ (L1-dominant)
- Run 4: $\lambda = 1 \times 10^{-5}$, $\alpha = 0.90$ (L1-dominant)

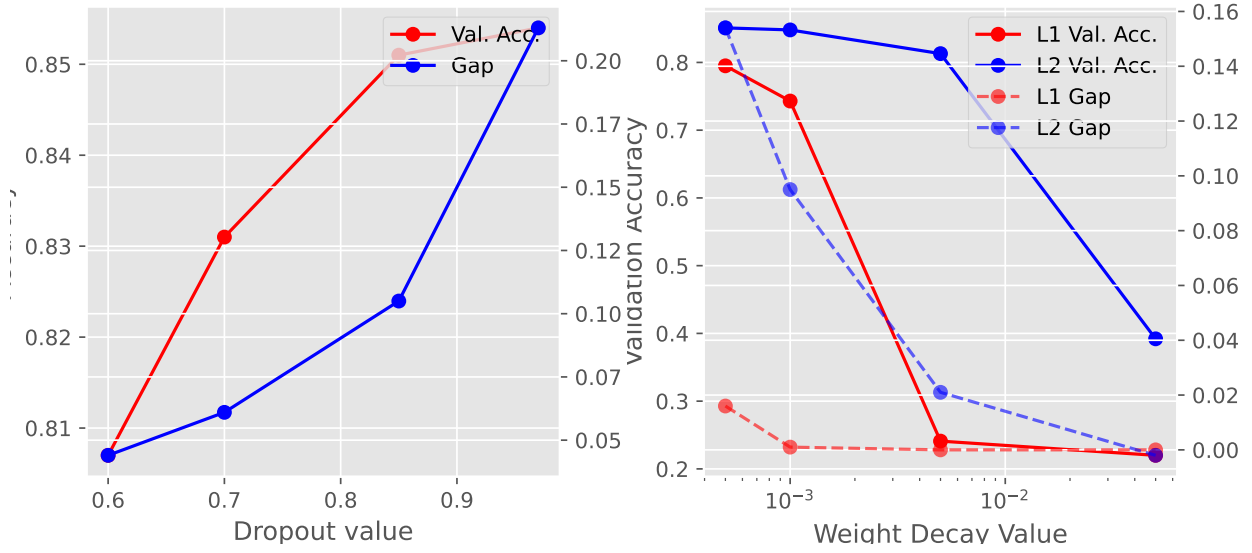
These configurations allow us to assess the impact of varying the balance between L1 and L2 regularisation while keeping the overall strength constant.

.

[Question 8 - Explain the experimental details (e.g. hyperparameters), discuss the results in terms of their generalisation performance and overfitting. Select and test the best performing model as part of this analysis.] Based on the results in Table 4, Run 2 ($\alpha = 0.75$) is the best-performing model. This configuration achieved a validation accuracy of 84.2%, a training error of 0.251, and a validation error of 0.495.

The critical metric for this selection is the validation error, which is the lowest of all four runs. While Run 1 ($\alpha = 0.9$) has a marginally higher validation accuracy (84.3%), its higher validation error (0.503 vs 0.495) indicates slightly worse generalisation. A lower validation error is the most direct measure of how well a model performs on unseen data, suggesting Run 2 is the least overfit model.

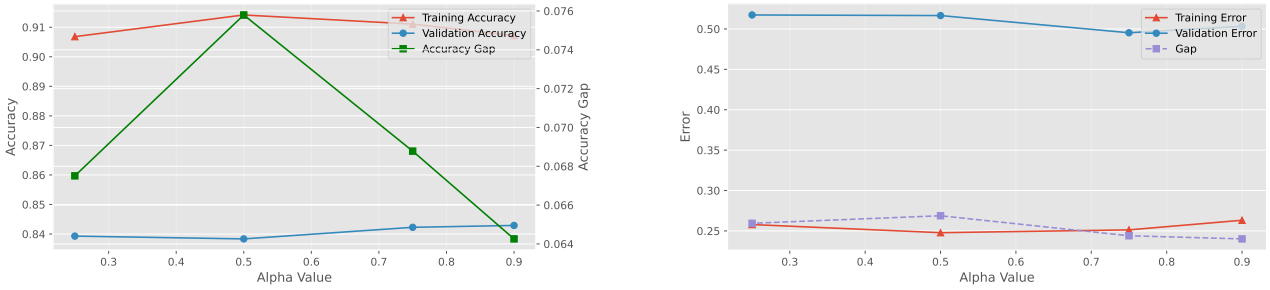
Overall, the results indicate that a strong L_1-bias (a high α value) is beneficial for this problem, as both Run 1 ($\alpha = 0.9$) and Run 2 ($\alpha = 0.75$) clearly outperformed the more balanced (Run 3, $\alpha = 0.5$) and L_2-biased (Run 4, $\alpha = 0.25$) configurations. Run 2 appears to hit the optimal point, leveraging the benefits of a strong L_1 penalty more



(a) Accuracy and error by inclusion probability.

(b) Accuracy and error by weight penalty.

Figure 4. Accuracy and error by regularisation strength of each method (Dropout and L1/L2 Regularisation).



(a) Validation and training accuracy, and generalisation gap for l1+l2 weight penalty.

(b) Training and validation error, and generalisation gap for l1+l2 weight penalty.

Figure 5. Training and validation performance across regularisation strengths for L1/L2 methods. The generalisation gap highlights overfitting effects across experiments.

effectively for generalisation than the other models. .

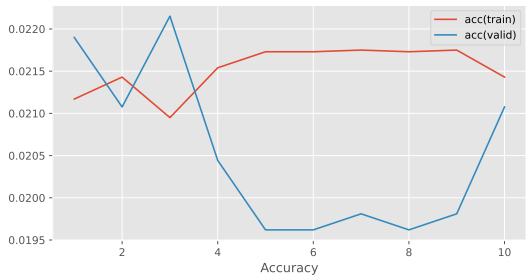
4.1. Custom Activation Function

[Question 9 - A colleague proposed using a custom activation function (which can be identified as CustomActivationLayer() in your Coursework_1.ipynb notebook and the mlp package). Run the prepared experiment in the "Problematic Training" cell in your Coursework_1.ipynb notebook, and display the results in Figure 6. Analyse the results. Is the model overfitting? Explain the cause of the problematic performance of the model?] The results in Figure 6 indicate that the model with the custom activation function struggles to learn effectively. The accuracy, and error curves behave very erratically. The training accuracy remains low, and the training error decreases, but not significantly, over epochs, suggesting that the model is not fitting the training data well. This poor performance is likely due to vanishing gradients, as evidenced by the gradient flow plot, which shows that gradients diminish across layers during backpropagation. This issue prevents effective weight updates, leading to stagnation in learning. Consequently, the model does not exhibit overfitting; instead, it fails to learn from the training data altogether. It is also worth noting that the experiment was only run for 10 epochs, which may not be sufficient for convergence, but the gradient issues suggest deeper problems with the activation function choice.

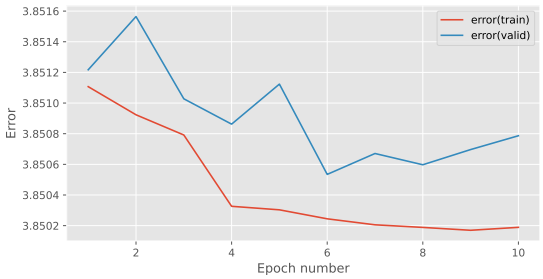
.

5. Conclusion

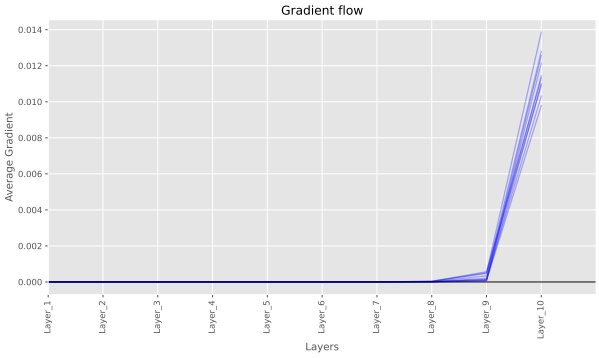
[Question 10 - Briefly draw your conclusions based on the results from the previous sections (what are the take-away messages?), discussing them in the context of the overall literature, and conclude your report with a recommendation for future directions] The analysis



(a) Validation and training accuracy, for custom activation layer.



(b) Training and validation error, and generalisation gap for custom activation layer.



(c) Training and validation gradients for custom activation layer.

Figure 6. Training and validation performance across regularisation strengths for custom activation layer.

presented in this report confirmed that increasing model capacity, either through width or depth, improves training fit but consistently leads to overfitting and poorer generalisation on the EMNIST dataset, as evidenced by the growing validation error. This aligns with the well-established bias-variance trade-off in the literature. Our experiments demonstrated that several regularisation techniques can effectively mitigate this. Both L2 regularisation (at 5×10^{-4}) and Dropout (at a 0.97 keep probability) successfully improved validation accuracy and reduced the generalisation gap compared to the baseline. Label smoothing also offered a modest improvement. While L1 regularisation alone performed poorly, our Elastic Net experiment found that a combination heavily biased towards L1 (a high α of 0.75) achieved the lowest validation error in its group, suggesting that the sparsity-inducing benefits of L1 are effective when balanced with L2's weight shrinkage. Finally, the failure of the custom activation function highlighted that poor performance is not always overfitting; in this case, it was a critical vanishing gradient problem that prevented the model from learning at all. Future work should involve a more comprehensive grid search over the Elastic Net hyperparameters (λ and α) and explore combinations of the most effective techniques, such as applying Dropout and L2 regularisation simultaneously, to potentially achieve further gains in generalisation. .

References

Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

Ng, Andrew Y. Feature selection, l1 vs. l2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 78, 2004.

Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958, 2014.

Zou, Hui and Hastie, Trevor. Zou h, hastie t. regularization and variable selection via the elastic net. j r statist soc b. 2005;67(2):301-20. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67:301 – 320, 04 2005. doi: 10.1111/j.1467-9868.2005.00503.x.